In [1]:
```
import pandas as pd
import numpy as np
from collections import Counter
import matplotlib.pyplot as plt
import seaborn as sns
import math
from scratch.linear_algebra import Vector,distance
import random
from collections import Counter
from typing import NamedTuple
```

In [2]:
```
# ==================================
# Import DataSet
# ==================================
df=pd.read_csv("test_dataSet.csv")
print ('Data Set:')
print (f"Columns\t:{df.shape[1]} \nRows\t:{df.shape[0]}")
df.head()
```

```
Data Set:
Columns :6
Rows    :100
```

Out[2]:

	No	Usia	BMI	Glukosa	Tekanan_Darah	Outcome
0	1	22	21	85	110	0
1	2	25	23	90	115	0
2	3	28	24	95	118	0
3	4	30	26	100	120	0
4	5	32	27	105	122	0

In [3]:
```
# ==================================
# Title Columns
# ==================================
print (df.columns)
print ("fixed")
df.columns =df.columns.str.strip().str.lower().str.replace(' ',"_")
print (df.columns)
```

```
Index(['No', 'Usia', 'BMI', 'Glukosa', 'Tekanan_Darah', 'Outcome'], dtype
='object')
fixed
Index(['no', 'usia', 'bmi', 'glukosa', 'tekanan_darah', 'outcome'], dtype
='object')
```

In [4]:
```
df.info ()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 6 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   no             100 non-null    int64
 1   usia           100 non-null    int64
 2   bmi            100 non-null    int64
 3   glukosa        100 non-null    int64
 4   tekanan_darah  100 non-null    int64
 5   outcome        100 non-null    int64
dtypes: int64(6)
memory usage: 4.8 KB
```

In [5]:
```
df.isnull().sum()
```

Out[5]:
```
no               0
usia             0
bmi              0
glukosa          0
tekanan_darah    0
outcome          0
dtype: int64
```

In [6]:
```
# ====================================
# Statistik Central Tedency
# ====================================
df.describe()
```

Out[6]:

	no	usia	bmi	glukosa	tekanan_darah	outcome
count	100.000000	100.000000	100.000000	100.000000	100.000000	100.000000
mean	50.500000	35.480000	30.040000	120.490000	127.060000	0.500000
std	29.011492	8.373214	6.365818	28.565889	11.730837	0.502519
min	1.000000	21.000000	19.000000	80.000000	105.000000	0.000000
25%	25.750000	28.750000	24.750000	96.750000	118.000000	0.000000
50%	50.500000	35.500000	29.000000	114.500000	125.500000	0.500000
75%	75.250000	42.000000	35.000000	140.500000	136.250000	1.000000
max	100.000000	52.000000	44.000000	195.000000	155.000000	1.000000

In [7]:
```
df.head (5)
```

Out[7]:

	no	usia	bmi	glukosa	tekanan_darah	outcome
0	1	22	21	85	110	0
1	2	25	23	90	115	0
2	3	28	24	95	118	0
3	4	30	26	100	120	0
4	5	32	27	105	122	0

In [8]:
```
# rescaling data
```

```
keys=['usia','usia','bmi','glukosa','tekanan_darah']
print (keys)
```

```
['usia', 'usia', 'bmi', 'glukosa', 'tekanan_darah']
```

In [9]:
```
for i in keys:
    min_=df[i].min()
    max_=df[i].max()
    val=[]
    for j in df[i]:
        new_val=(j-min_)/(max_-min_)
        val.append(new_val)
    print (val)
    df[i]=val
```

```
[np.float64(0.03225806451612903), np.float64(0.12903225806451613), np.floa
t64(0.22580645161290322), np.float64(0.2903225806451613), np.float64(0.354
8387096774194), np.float64(0.45161290322580644), np.float64(0.548387096774
1935), np.float64(0.6129032258064516), np.float64(0.6774193548387096), n
p.float64(0.7741935483870968), np.float64(0.06451612903225806), np.float64
(0.16129032258064516), np.float64(0.25806451612903225), np.float64(0.32258
06451612903), np.float64(0.41935483870967744), np.float64(0.51612903225806
45), np.float64(0.5806451612903226), np.float64(0.6451612903225806), np.fl
oat64(0.7419354838709677), np.float64(0.8387096774193549), np.float64(0.09
67741935483871), np.float64(0.1935483870967742), np.float64(0.290322580645
1613), np.float64(0.3870967741935484), np.float64(0.4838709677419355), n
p.float64(0.5483870967741935), np.float64(0.6129032258064516), np.float64
(0.7096774193548387), np.float64(0.8064516129032258), np.float64(0.8709677
419354839), np.float64(0.03225806451612903), np.float64(0.1290322580645161
3), np.float64(0.22580645161290322), np.float64(0.3225806451612903), np.fl
oat64(0.41935483870967744), np.float64(0.5161290322580645), np.float64(0.5
806451612903226), np.float64(0.6774193548387096), np.float64(0.77419354838
70968), np.float64(0.9032258064516129), np.float64(0.06451612903225806), n
p.float64(0.16129032258064516), np.float64(0.25806451612903225), np.float6
4(0.3548387096774194), np.float64(0.45161290322580644), np.float64(0.54838
70967741935), np.float64(0.6451612903225806), np.float64(0.741935483870967
7), np.float64(0.8387096774193549), np.float64(0.9354838709677419), np.flo
at64(0.03225806451612903), np.float64(0.0967741935483871), np.float64(0.19
35483870967742), np.float64(0.2903225806451613), np.float64(0.387096774193
5484), np.float64(0.4838709677419355), np.float64(0.5806451612903226), n
p.float64(0.6774193548387096), np.float64(0.7741935483870968), np.float64
(0.8709677419354839), np.float64(0.0), np.float64(0.06451612903225806), n
p.float64(0.16129032258064516), np.float64(0.25806451612903225), np.float6
4(0.3548387096774194), np.float64(0.45161290322580644), np.float64(0.54838
70967741935), np.float64(0.6451612903225806), np.float64(0.741935483870967
7), np.float64(0.8387096774193549), np.float64(0.12903225806451613), np.fl
oat64(0.22580645161290322), np.float64(0.3225806451612903), np.float64(0.4
1935483870967744), np.float64(0.5161290322580645), np.float64(0.6129032258
064516), np.float64(0.7096774193548387), np.float64(0.8064516129032258), n
p.float64(0.9032258064516129), np.float64(1.0), np.float64(0.0322580645161
2903), np.float64(0.0967741935483871), np.float64(0.1935483870967742), n
p.float64(0.2903225806451613), np.float64(0.3870967741935484), np.float64
(0.4838709677419355), np.float64(0.5806451612903226), np.float64(0.6774193
548387096), np.float64(0.7741935483870968), np.float64(0.870967741935483
9), np.float64(0.06451612903225806), np.float64(0.16129032258064516), np.f
loat64(0.25806451612903225), np.float64(0.3548387096774194), np.float64
(0.45161290322580644), np.float64(0.5483870967741935), np.float64(0.645161
2903225806), np.float64(0.7419354838709677), np.float64(0.838709677419354
9), np.float64(0.9354838709677419)]
[np.float64(0.03225806451612903), np.float64(0.12903225806451613), np.floa
t64(0.22580645161290322), np.float64(0.2903225806451613), np.float64(0.354
8387096774194), np.float64(0.45161290322580644), np.float64(0.548387096774
1935), np.float64(0.6129032258064516), np.float64(0.6774193548387096), n
p.float64(0.7741935483870968), np.float64(0.06451612903225806), np.float64
(0.16129032258064516), np.float64(0.25806451612903225), np.float64(0.32258
06451612903), np.float64(0.41935483870967744), np.float64(0.51612903225806
45), np.float64(0.5806451612903226), np.float64(0.6451612903225806), np.fl
oat64(0.7419354838709677), np.float64(0.8387096774193549), np.float64(0.09
67741935483871), np.float64(0.1935483870967742), np.float64(0.290322580645
1613), np.float64(0.3870967741935484), np.float64(0.4838709677419355), n
p.float64(0.5483870967741935), np.float64(0.6129032258064516), np.float64
(0.7096774193548387), np.float64(0.8064516129032258), np.float64(0.8709677
419354839), np.float64(0.03225806451612903), np.float64(0.1290322580645161
3), np.float64(0.22580645161290322), np.float64(0.3225806451612903), np.fl
oat64(0.41935483870967744), np.float64(0.5161290322580645), np.float64(0.5
```

```
806451612903226), np.float64(0.6774193548387096), np.float64(0.77419354838
70968), np.float64(0.9032258064516129), np.float64(0.06451612903225806), n
p.float64(0.16129032258064516), np.float64(0.25806451612903225), np.float6
4(0.3548387096774194), np.float64(0.45161290322580644), np.float64(0.54838
70967741935), np.float64(0.6451612903225806), np.float64(0.741935483870967
7), np.float64(0.8387096774193549), np.float64(0.9354838709677419), np.flo
at64(0.03225806451612903), np.float64(0.0967741935483871), np.float64(0.19
35483870967742), np.float64(0.2903225806451613), np.float64(0.387096774193
5484), np.float64(0.4838709677419355), np.float64(0.5806451612903226), n
p.float64(0.6774193548387096), np.float64(0.7741935483870968), np.float64
(0.8709677419354839), np.float64(0.0), np.float64(0.06451612903225806), n
p.float64(0.16129032258064516), np.float64(0.25806451612903225), np.float6
4(0.3548387096774194), np.float64(0.45161290322580644), np.float64(0.54838
70967741935), np.float64(0.6451612903225806), np.float64(0.741935483870967
7), np.float64(0.8387096774193549), np.float64(0.12903225806451613), np.fl
oat64(0.22580645161290322), np.float64(0.3225806451612903), np.float64(0.4
1935483870967744), np.float64(0.5161290322580645), np.float64(0.6129032258
064516), np.float64(0.7096774193548387), np.float64(0.8064516129032258), n
p.float64(0.9032258064516129), np.float64(1.0), np.float64(0.0322580645161
2903), np.float64(0.0967741935483871), np.float64(0.1935483870967742), n
p.float64(0.2903225806451613), np.float64(0.3870967741935484), np.float64
(0.4838709677419355), np.float64(0.5806451612903226), np.float64(0.6774193
548387096), np.float64(0.7741935483870968), np.float64(0.870967741935483
9), np.float64(0.06451612903225806), np.float64(0.16129032258064516), np.f
loat64(0.25806451612903225), np.float64(0.3548387096774194), np.float64
(0.45161290322580644), np.float64(0.5483870967741935), np.float64(0.645161
2903225806), np.float64(0.7419354838709677), np.float64(0.838709677419354
9), np.float64(0.9354838709677419)]
[np.float64(0.08), np.float64(0.16), np.float64(0.2), np.float64(0.28), n
p.float64(0.32), np.float64(0.4), np.float64(0.48), np.float64(0.56), np.f
loat64(0.6), np.float64(0.68), np.float64(0.12), np.float64(0.2), np.float
64(0.24), np.float64(0.32), np.float64(0.36), np.float64(0.44), np.float64
(0.52), np.float64(0.64), np.float64(0.72), np.float64(0.8), np.float64
(0.16), np.float64(0.2), np.float64(0.28), np.float64(0.32), np.float64
(0.4), np.float64(0.48), np.float64(0.56), np.float64(0.64), np.float64
(0.76), np.float64(0.84), np.float64(0.12), np.float64(0.16), np.float64
(0.24), np.float64(0.32), np.float64(0.4), np.float64(0.48), np.float64
(0.56), np.float64(0.68), np.float64(0.76), np.float64(0.88), np.float64
(0.08), np.float64(0.16), np.float64(0.2), np.float64(0.28), np.float64
(0.36), np.float64(0.44), np.float64(0.6), np.float64(0.72), np.float64
(0.8), np.float64(0.92), np.float64(0.04), np.float64(0.12), np.float64
(0.2), np.float64(0.28), np.float64(0.36), np.float64(0.44), np.float64
(0.52), np.float64(0.64), np.float64(0.72), np.float64(0.84), np.float64
(0.0), np.float64(0.08), np.float64(0.16), np.float64(0.24), np.float64
(0.32), np.float64(0.4), np.float64(0.48), np.float64(0.6), np.float64(0.6
8), np.float64(0.8), np.float64(0.12), np.float64(0.2), np.float64(0.28),
np.float64(0.36), np.float64(0.44), np.float64(0.56), np.float64(0.64), n
p.float64(0.76), np.float64(0.88), np.float64(0.96), np.float64(0.08), n
p.float64(0.16), np.float64(0.24), np.float64(0.32), np.float64(0.4), np.f
loat64(0.48), np.float64(0.56), np.float64(0.68), np.float64(0.76), np.flo
at64(0.88), np.float64(0.04), np.float64(0.12), np.float64(0.2), np.float6
4(0.28), np.float64(0.36), np.float64(0.44), np.float64(0.56), np.float64
(0.72), np.float64(0.84), np.float64(1.0)]
[np.float64(0.043478260869565216), np.float64(0.08695652173913043), np.flo
at64(0.13043478260869565), np.float64(0.17391304347826086), np.float64(0.2
1739130434782608), np.float64(0.2608695652173913), np.float64(0.3478260869
5652173), np.float64(0.43478260869565216), np.float64(0.5217391304347826),
np.float64(0.6086956521739131), np.float64(0.06956521739130435), np.float6
4(0.10434782608695652), np.float64(0.1565217391304348), np.float64(0.24347
826086956523), np.float64(0.2782608695652174), np.float64(0.33043478260869
```

563), np.float64(0.391304347826087), np.float64(0.5043478260869565), np.fl
oat64(0.591304347826087), np.float64(0.6956521739130435), np.float64(0.078
2608695652174), np.float64(0.12173913043478261), np.float64(0.191304347826
08696), np.float64(0.23478260869565218), np.float64(0.30434782608695654),
np.float64(0.3652173913043478), np.float64(0.4782608695652174), np.float64
(0.5652173913043478), np.float64(0.6521739130434783), np.float64(0.7826086
95652174), np.float64(0.06086956521739131), np.float64(0.0956521739130434
8), np.float64(0.14782608695652175), np.float64(0.21739130434782608), np.f
loat64(0.2782608695652174), np.float64(0.3391304347826087), np.float64(0.4
1739130434782606), np.float64(0.5391304347826087), np.float64(0.6260869565
217392), np.float64(0.8260869565217391), np.float64(0.034782608695652174),
np.float64(0.11304347826086956), np.float64(0.16521739130434782), np.float
64(0.20869565217391303), np.float64(0.26956521739130435), np.float64(0.330
43478260869563), np.float64(0.4956521739130435), np.float64(0.6), np.float
64(0.7391304347826086), np.float64(0.8695652173913043), np.float64(0.01739
1304347826087), np.float64(0.06956521739130435), np.float64(0.130434782608
69565), np.float64(0.1826086956521739), np.float64(0.25217391304347825), n
p.float64(0.3217391304347826), np.float64(0.4), np.float64(0.5217391304347
826), np.float64(0.6086956521739131), np.float64(0.8), np.float64(0.0), n
p.float64(0.05217391304347826), np.float64(0.10434782608695652), np.float6
4(0.1565217391304348), np.float64(0.22608695652173913), np.float64(0.29565
21739130435), np.float64(0.3739130434782609), np.float64(0.504347826086956
5), np.float64(0.5826086956521739), np.float64(0.7130434782608696), np.flo
at64(0.08695652173913043), np.float64(0.1391304347826087), np.float64
(0.2), np.float64(0.2608695652173913), np.float64(0.33043478260869563), n
p.float64(0.45217391304347826), np.float64(0.5565217391304348), np.float64
(0.6782608695652174), np.float64(0.8347826086956521), np.float64(0.9565217
391304348), np.float64(0.02608695652173913), np.float64(0.078260869565217
4), np.float64(0.12173913043478261), np.float64(0.19130434782608696), np.f
loat64(0.24347826086956523), np.float64(0.3130434782608696), np.float64
(0.391304347826087), np.float64(0.5130434782608696), np.float64(0.63478260
86956522), np.float64(0.782608695652174), np.float64(0.008695652173913044
4), np.float64(0.06086956521739131), np.float64(0.11304347826086956), np.f
loat64(0.1826086956521739), np.float64(0.25217391304347825), np.float64
(0.33043478260869563), np.float64(0.43478260869565216), np.float64(0.61739
13043478261), np.float64(0.7652173913043478), np.float64(1.0)]
[np.float64(0.1), np.float64(0.2), np.float64(0.26), np.float64(0.3), np.f
loat64(0.34), np.float64(0.4), np.float64(0.5), np.float64(0.6), np.float6
4(0.66), np.float64(0.7), np.float64(0.14), np.float64(0.22), np.float64
(0.28), np.float64(0.32), np.float64(0.38), np.float64(0.46), np.float64
(0.54), np.float64(0.62), np.float64(0.68), np.float64(0.74), np.float64
(0.18), np.float64(0.24), np.float64(0.3), np.float64(0.36), np.float64
(0.42), np.float64(0.48), np.float64(0.58), np.float64(0.64), np.float64
(0.72), np.float64(0.8), np.float64(0.12), np.float64(0.2), np.float64(0.2
6), np.float64(0.32), np.float64(0.38), np.float64(0.44), np.float64(0.5
2), np.float64(0.6), np.float64(0.68), np.float64(0.86), np.float64(0.08),
np.float64(0.22), np.float64(0.28), np.float64(0.34), np.float64(0.4), n
p.float64(0.46), np.float64(0.56), np.float64(0.66), np.float64(0.78), n
p.float64(0.9), np.float64(0.06), np.float64(0.14), np.float64(0.24), np.f
loat64(0.3), np.float64(0.36), np.float64(0.44), np.float64(0.52), np.floa
t64(0.62), np.float64(0.7), np.float64(0.82), np.float64(0.0), np.float64
(0.1), np.float64(0.18), np.float64(0.26), np.float64(0.32), np.float64
(0.4), np.float64(0.48), np.float64(0.58), np.float64(0.66), np.float64
(0.76), np.float64(0.16), np.float64(0.24), np.float64(0.32), np.float64
(0.38), np.float64(0.46), np.float64(0.56), np.float64(0.64), np.float64
(0.74), np.float64(0.84), np.float64(0.94), np.float64(0.08), np.float64
(0.16), np.float64(0.22), np.float64(0.3), np.float64(0.36), np.float64
(0.44), np.float64(0.52), np.float64(0.62), np.float64(0.7), np.float64
(0.82), np.float64(0.04), np.float64(0.12), np.float64(0.2), np.float64
(0.28), np.float64(0.38), np.float64(0.46), np.float64(0.54), np.float64

```
(0.66), np.float64(0.78), np.float64(1.0)]
```

In [10]:
```
# ======================================
# buid model
# ======================================
# set feature
x =df[df.columns[:-1]]
y= df[df.columns[-1]]

def varible(x,y):
    points =x.values.tolist()
    keys =y.values.tolist()
    return points,keys

points,keys=varible(x,y )
```

In [11]:
```
# ================================
# split data test and train
# ================================

def shuffle_index(data:list[int],prob:float):
    data=data[:]
    random.shuffle(data)
    cut=int (len(data)*prob)
    return data[:cut],data[cut:]

def split_train_test(xs:list[Vector],ys:list[float],test_pct:float):
    # index
    idx=[i for i in range(len(xs))]
    train_pct=1 -test_pct
    idx_train,idx_test=shuffle_index(idx,train_pct)
    return (
        # train
        [xs[i] for i in idx_train],
        [ys [i] for i in  idx_train],
        # test
        [xs[i] for i in idx_test],
        [ys [i] for i in idx_test]
    )
```

In [12]:
```
# K-Nearst model with scracth

# vote fungstion
def vote(labels:list[str]):
    count =Counter(labels)
    winner, winner_val =count.most_common(1)[0]
    # if winner is not 1
    num_winner =len([i for i in count.values() if i == winner_val])
    if num_winner==1:
        return winner
    else :
        return vote(labels[:-1])
```

In [13]:
```
class labelPoints(NamedTuple):
    point:Vector
    label:str

def k_near(k:int,labelpoint:list[labelPoints],new_point:Vector):
```

```
    by_distance=sorted(labelpoint,key=lambda lp: distance(lp.point,new_po
    label_k=[lp.label for lp in by_distance[:k]]
    return vote(label_k)
```

In [14]:
```
points_train,keys_train,points_test,keys_test =split_train_test (points,k
```

In [15]:
```
lp_train = [
    labelPoints(p, l)
    for p, l in zip(points_train, keys_train)
]
lp_test =[
    labelPoints(p,l) for p,l in zip(points_test,keys_test)
]
```

In [16]:
```
# full varible
num_corect=0
for test in lp_test:
    predict = k_near(2,lp_train,test.point)
    actual =test.label
    if predict==actual:
        num_corect +=1

persentage=num_corect/len(lp_test)
print(f"accuracy :{persentage}")
```

```
accuracy :1.0
```